Understanding Modern Language Models

A Learner's Guide to Transformer Architecture, Pre-training vs. Fine-tuning, Tokenisation, and Embeddings

Structured study notes

October 29, 2026

Contents

1 Introduction and Learning Path **4**

2 Prerequisite Concepts **4**

2.1 AI, Machine Learning, and Deep Learning 4

2.2 Neural Networks and Parameters 4

2.3 Vectors and Matrices 4

3 Tokenisation: Converting Text into Model Inputs **5**

3.1 Why Tokenisation Is Needed 5

3.2 The Tokenisation Pipeline 5

3.3 Character, Word, and Subword Tokenisation 5

3.4 Common Tokenisation Algorithms 6

3.5 Token IDs and Embedding Lookup 6

3.6 Special Tokens 6

3.7 Context Windows and Cost 6

3.8 Tokenisation for Indian Languages and Mixed Text 6

3.9 Limitations 7

4 Embeddings: Representing Information as Vectors **7**

4.1 Definition 7

4.2 Token Embeddings and Contextual Representations 7

4.3 Embedding Matrix Mathematics 7

4.4 Similarity: Cosine Similarity and Euclidean Distance 7

4.5 How Embeddings Are Learned 8

4.6 Types of Embeddings 8

4.7 Semantic Search and RAG 8

4.8 Embedding Limitations 9

5 Transformer Architecture: The Full Picture **9**
5.1 What Is a Transformer? . . . 9
5.2 Overall Data Flow . . . 9
5.3 Step 1: Token Embeddings and Positional Information . . . 10
5.4 Step 2: Query, Key, and Value Projections . . . 10
5.5 Step 3: Attention Scores . . . 10
5.6 Step 4: Scaling and Softmax . . . 10
5.7 Step 5: Weighted Values . . . 11
5.8 Step 6: Causal Masking . . . 11
5.9 Step 7: Multi-Head Attention . . . 11
5.10 Step 8: Residual Connections . . . 12
5.11 Step 9: Layer Normalisation . . . 12
5.12 Step 10: Feed-Forward Network . . . 12
5.13 Step 11: Stacking Blocks . . . 12
5.14 Step 12: Output Projection and Token Selection . . . 12
5.15 Encoder, Decoder, and Encoder–Decoder . . . 13
5.16 Computational Cost . . . 13

6 Pre-training: Learning Broad Patterns **13**
6.1 Definition . . . 13
6.2 Next-Token Prediction . . . 13
6.3 Other Pre-training Objectives . . . 14
6.4 A Simplified Pre-training Pipeline . . . 14
6.5 What Does the Model Learn? . . . 14
6.6 Data Quality . . . 14
6.7 Why a Pre-trained Model May Not Be Ready for Users . . . 14

7 Fine-tuning: Adapting a Pre-trained Model **15**
7.1 Definition . . . 15
7.2 Supervised Fine-tuning . . . 15
7.3 Preference Optimisation and Alignment . . . 15
7.4 Full Fine-tuning . . . 15
7.5 Parameter-Efficient Fine-tuning and LoRA . . . 15
7.6 Fine-tuning Data Quality . . . 16
7.7 Does Fine-tuning Add New Facts? . . . 16

8 Pre-training vs. Fine-tuning: Direct Comparison **16**
8.1 A Simple Analogy . . . 16
8.2 Prompting vs. RAG vs. Fine-tuning . . . 17

9 Worked Enterprise Example: A Policy Assistant **17**
9.1 Possible Architecture . . . 17
9.2 Where Fine-tuning Helps . . . 17
9.3 Possible Failure Modes . . . 17

10 Practical Exercises **18**
10.1 Exercise 1: Inspect Tokenisation . . . 18
10.2 Exercise 2: Compare Sentence Embeddings . . . 18
10.3 Exercise 3: Draw a Transformer Block . . . 18
10.4 Exercise 4: Choose an Adaptation Method . . . 18
10.5 Exercise 5: Build an Evaluation Set . . . 18

11 Common Misunderstandings **19**

12 Glossary **19**

13 Self-Assessment Questions **20**

14 Conclusion **21**

1 Introduction and Learning Path

This document is written from the perspective of a learner who wants to understand modern language models step by step: what each component means, how it works, what the important mathematics is saying, and how the ideas are used in practical systems. The language is intentionally direct and conversational, with examples relevant to an Indian academic or professional setting.

The four main topics are:

1. Transformer architecture, from token input through attention and output generation.
2. Pre-training versus fine-tuning, including objectives, data, costs, and limitations.
3. Tokenisation, including token IDs, subwords, special tokens, and context windows.
4. Embeddings, including vector representations, similarity, semantic search, and RAG.

These topics connect into one pipeline: tokenisation converts text to IDs; an embedding layer maps IDs to vectors; Transformer layers contextualise those vectors; training teaches the model patterns; and fine-tuning can adapt a pre-trained model for a particular task.

2 Prerequisite Concepts

2.1 AI, Machine Learning, and Deep Learning

Artificial Intelligence (AI) is the broad field of building systems that perform tasks associated with intelligent behaviour. Machine Learning (ML) is a part of AI in which a system learns patterns from data. Deep Learning is a part of ML that uses neural networks with multiple layers.

A language model assigns probabilities to token sequences and can generate or analyse text. Many modern language models use deep neural networks based on the Transformer architecture.

2.2 Neural Networks and Parameters

A neural network is a parameterised mathematical function. A simple layer can be written as

$$h = \sigma(Wx + b),$$

where x is the input vector, W is a learned weight matrix, b is a learned bias, σ is a nonlinear activation function, and h is the output representation.

Three terms should not be confused:

- **Parameters** are learned values inside a model, such as weights and biases.
- **Hyperparameters** are choices such as learning rate, batch size, number of layers, and training steps.
- **Activations** are intermediate values produced as data passes through the network.

2.3 Vectors and Matrices

A vector is an ordered list of numbers; a matrix is a rectangular arrangement of numbers. If $X \in \mathbb{R}^{n \times d}$, it can represent n tokens, each represented by d numbers. This notation appears

throughout Transformer mathematics.

3 Tokenisation: Converting Text into Model Inputs

3.1 Why Tokenisation Is Needed

A neural network operates on numerical values, not directly on words as humans see them. Tokenisation converts a text string into discrete units called *tokens*. Each token is assigned an integer ID from a vocabulary.

For example, the sentence "The model reads a document." might be split into units resembling

$$[\texttt{The},\ \texttt{model},\ \texttt{reads},\ \texttt{a},\ \texttt{document},\ \texttt{.}]$$

and then converted into integer IDs. This is only an illustration: a real tokenizer may split the same sentence differently. A token can be a whole word, a word fragment, punctuation, whitespace-associated text, or another unit. One token is not necessarily one word or one character.

3.2 The Tokenisation Pipeline

A simplified pipeline is:

1. Receive the text.
2. Apply the tokenizer's normalisation and pre-tokenisation rules.
3. Split text into units according to its vocabulary and algorithm.
4. Map each unit to an integer token ID.
5. Pass the ID sequence to the model.

At the output stage, token IDs can be decoded back into text.

3.3 Character, Word, and Subword Tokenisation

Approach	**Idea**	**Trade-off**
Character-level	Each character or byte-like unit is a token	Small basic vocabulary and flexible handling of new words, but sequences can become long.
Word-level	Each whole word is a token	Easy to understand, but vocabulary can become huge and unseen words are difficult.
Subword-level	Common words and fragments are tokens	Balances vocabulary size and sequence length; rare words can be composed from familiar pieces.

Modern language models commonly use subword or related methods.

3.4 Common Tokenisation Algorithms

Byte Pair Encoding (BPE) starts with small units and repeatedly merges frequent adjacent pairs when building the vocabulary. The learned merge rules are then used to split text.

WordPiece learns a subword vocabulary using a criterion related to how well candidate pieces represent the training data. It is associated with BERT-style tokenisation, though implementations differ.

Unigram tokenisation begins with a larger set of possible pieces and selects a vocabulary using a probabilistic objective. SentencePiece is a toolkit that supports Unigram and BPE.

These are algorithm families, not identical implementations. Tokenizer details matter when reproducing results or estimating token usage.

3.5 Token IDs and Embedding Lookup

Suppose a tokenizer produces IDs $[17, 205, 91, 8]$. These numbers are labels, not measurements: ID 205 is not inherently more important than ID 17.

The model has an embedding matrix $E \in \mathbb{R}^{V \times d}$, where V is the vocabulary size and d is the embedding dimension. For token ID i, the model looks up row i:

$$e_i = E[i].$$

The result is a vector of d numbers. This is how discrete token IDs become continuous representations for neural-network computation.

3.6 Special Tokens

Tokenizers may include special tokens for beginning or end of sequence, padding, separators, chat roles, or multimodal content such as images and audio. The exact tokens and their meanings differ by model family. Do not assume that every model uses the same special-token scheme.

3.7 Context Windows and Cost

A model's *context window* is the amount of tokenised input and generated content it can process in one context. A long document can use many tokens; page count or word count is only a rough guide. Different scripts, languages, punctuation patterns, code, and formatting can have different tokenisation behaviour.

API usage is often priced by input and output tokens. A long prompt and long answer can therefore cost more than a short interaction. Use the tokenizer associated with the selected model for accurate token counts.

3.8 Tokenisation for Indian Languages and Mixed Text

Hindi, Tamil, Bengali, other Indian languages, English, Hinglish, transliteration, emojis, names, and code may be tokenised differently depending on the vocabulary and training. A sentence in a regional script may use a different number of tokens than a similar English sentence. This can affect latency, cost, and model quality. Evaluate the actual languages and writing styles expected in the application instead of assuming English results will generalise.

3.9 Limitations

Token boundaries do not always match words; rare names may use several tokens; different tokenizers can produce different IDs; and tokenisation alone does not determine multilingual ability. Token IDs are categorical labels, not numerical measurements.

4 Embeddings: Representing Information as Vectors

4.1 Definition

An embedding is a numerical vector representation of an item such as a token, sentence, document, image, or product. It gives a neural network a way to process information through mathematical operations. Each dimension is a learned number; it is usually not correct to assume that one dimension explicitly means "happiness" or "past tense". Information is typically distributed across the vector.

4.2 Token Embeddings and Contextual Representations

A **token embedding** is the vector looked up for a token ID before the Transformer processes it. A **contextual representation** is the vector after the token has passed through Transformer layers and incorporated information from surrounding tokens.

Consider:

> I deposited money in the bank.
> We sat on the bank of the river.

If "bank" is represented by the same token ID in both sentences, the initial lookup may be the same. After attention processes the context, its representation can differ because the surrounding words suggest different meanings.

4.3 Embedding Matrix Mathematics

Let the vocabulary have V IDs and each vector have dimension d:

$$E \in \mathbb{R}^{V \times d}.$$

The embedding for token ID i is the i-th row:

$$e_i = E_{i,:} \in \mathbb{R}^d.$$

A sequence of n tokens becomes a matrix $X \in \mathbb{R}^{n \times d}$. Each row represents one token. The Transformer processes this matrix, usually with positional information or position-dependent operations.

4.4 Similarity: Cosine Similarity and Euclidean Distance

A common vector similarity measure is

$$\cos(a, b) = \frac{a^\top b}{\|a\|_2 \|b\|_2}.$$

For non-zero vectors it lies between -1 and 1. Values nearer to 1 indicate vectors pointing in similar directions. Its usefulness for semantic similarity depends on how the embedding model was trained.

Euclidean distance is

$$d(a, b) = \sqrt{\sum_{i=1}^{d} (a_i - b_i)^2}.$$

A smaller distance means the vectors are closer in Euclidean space. Cosine similarity and Euclidean distance are not interchangeable in every setting, especially when vector norms differ.

For a simple example, $a = [1, 2]$ and $b = [2, 4]$ have cosine similarity 1, because they point in the same direction. A vector such as $c = [-1, 1]$ points in a different direction. Real embeddings have many more dimensions and their useful relationships are learned from data.

4.5 How Embeddings Are Learned

Embedding vectors are commonly learned as part of model training. When an embedding contributes to a lower training loss, its parameters can be updated by gradient descent. The model is not necessarily given a dictionary that explicitly labels synonyms; useful relationships can emerge from patterns in the data and the training objective.

Not all embedding models are designed for the same purpose. A language-model token embedding is not automatically the best vector for document search. Sentence and retrieval embedding models are often trained specifically so relevant texts have useful similarity relationships.

4.6 Types of Embeddings

Type	**Represents**	**Common use**
Token/word	Token or word-level unit	Language modelling and contextual representations
Sentence	Sentence or short text	Semantic similarity, clustering, question matching
Passage/document	Paragraph, chunk, or document	Semantic search and RAG
Image	Image or image region	Image retrieval and similarity
Multimodal	Text and other data in related vector spaces	Matching text descriptions with images or supported media

A single vector compresses information and cannot preserve every detail of a long document. For retrieval, documents are often split into chunks, each embedded separately, with metadata such as source, section, and date.

4.7 Semantic Search and RAG

Keyword search looks for matching terms. Semantic search uses embeddings to find content related in meaning, even if the wording differs. An employee asking "How do I get my travel money back?" may need a policy section titled "Travel expense reimbursement". Embeddings can help connect these expressions.

A simple pipeline is:

1. Split approved documents into passages.
2. Create an embedding for each passage.
3. Store vectors and metadata in an index.
4. Embed the user's query with a compatible encoder.
5. Rank passages by similarity or a learned retrieval score.
6. Return relevant passages to the user or pass them to a language model.

Retrieval-Augmented Generation (RAG) uses retrieved passages as context for a generative model. Embeddings do not themselves generate the final answer and do not enforce access permissions. The application must check permissions and ensure that documents are current.

4.8 Embedding Limitations

Similarity is a signal, not proof of relevance. Long passages can lose detail when compressed into one vector. Domain-specific terms may be poorly represented. Vectors from different embedding models should not be compared casually because they may inhabit incompatible spaces. Embedding indexes also require privacy controls and access restrictions.

5 Transformer Architecture: The Full Picture

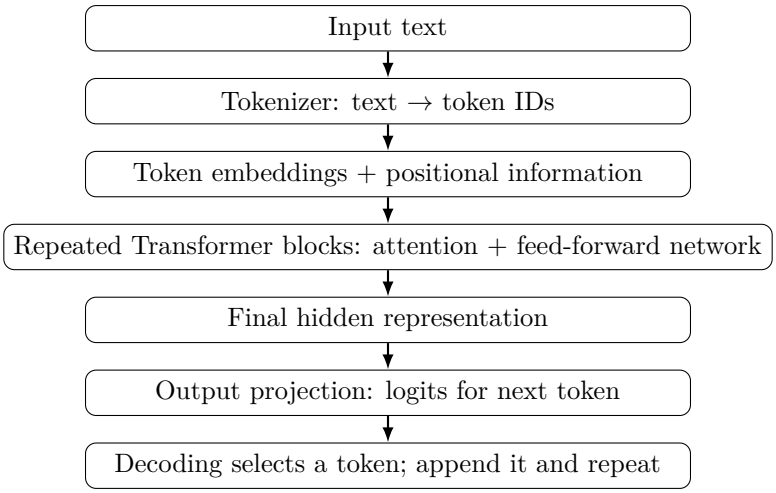
5.1 What Is a Transformer?

A Transformer is a neural-network architecture built around attention and feed-forward layers. Attention allows a token to combine information from other relevant token positions. Transformers became important because they model relationships across a sequence and permit much of the training computation to be parallelised.

Earlier recurrent neural networks (RNNs) and LSTMs process sequence elements recurrently. Transformers provide a different way to model dependencies, including long-distance relationships. Related Transformer designs are used in language, vision, audio, and multimodal systems.

5.2 Overall Data Flow

For a simplified decoder-only language model, the flow is:



This is a conceptual diagram. Real implementations also include details such as normalisation placement, masking, caching, and model-specific components.

5.3 Step 1: Token Embeddings and Positional Information

After tokenisation, token IDs are mapped to vectors. For n tokens and hidden dimension d, the input representation is $X \in \mathbb{R}^{n \times d}$.

Attention does not automatically represent token order in every architecture, so positional information is added or otherwise incorporated. A traditional approach is

$$X_i = E_i + P_i,$$

where E_i is the token embedding and P_i is a positional vector. Other approaches, such as rotary position embeddings, modify attention computations. The implementation depends on the model.

5.4 Step 2: Query, Key, and Value Projections

Self-attention first transforms the input matrix X:

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V.$$

The matrices W^Q, W^K, W^V are learned parameters. The letters stand for query, key, and value.

An analogy is a library: a query represents what a token is looking for; keys provide representations to match against; values contain the information that can be passed to the token. These are mathematical representations, not literal questions written by a human.

5.5 Step 3: Attention Scores

The query and key vectors are compared using a matrix product:

$$S = QK^\top.$$

For n tokens, standard full self-attention creates an $n \times n$ score matrix. Each score indicates how strongly a query position matches a key position according to the learned projections. Scores are not yet probabilities and do not necessarily correspond to a human-readable notion of importance.

5.6 Step 4: Scaling and Softmax

Scaled dot-product attention is

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right),$$

where d_k is the key dimension. Dividing by $\sqrt{d_k}$ helps keep dot-product magnitudes in a useful range as dimensions grow. Softmax converts each row of allowed scores into weights that sum to one:

$$\text{softmax}(s_i) = \frac{e^{s_i}}{\sum_j e^{s_j}}.$$

A larger score receives a larger weight relative to the other scores in the same row.

5.7 Step 5: Weighted Values

The attention output is

$$H = AV.$$

Each output row is a weighted combination of value vectors. If a token gives another position a high weight, that position's value contributes more strongly. Attention therefore allows token representations to incorporate context from other positions.

Consider "The student left the book on the table because it was heavy." A model may learn to relate "it" to "book". Attention offers a mechanism for such relationships, but attention weights alone are not a complete explanation of the model's reasoning; many layers and operations contribute to the final output.

5.8 Step 6: Causal Masking

A GPT-style decoder is trained to predict the next token. When processing a training sequence, position i must not access future tokens, because those future tokens include the target it is meant to predict. A causal mask blocks attention to future positions.

A simplified masked score matrix is

$$S_{\text{masked}} = S + M,$$

where $M_{ij} = 0$ for allowed positions and a very large negative value (conceptually $-\infty$) for blocked positions. After softmax, blocked positions receive zero weight.

For four tokens, the allowed pattern is:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

where 1 means allowed and 0 means blocked. This is a mask, not the attention output. Encoder-style models may use different masking rules.

5.9 Step 7: Multi-Head Attention

Transformers typically use several attention heads in parallel. Each head has learned projections and can represent different patterns of relationships:

$$H_i = \text{Attention}(Q_i, K_i, V_i).$$

The outputs are concatenated and projected:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(H_1, \ldots, H_h)W^O.$$

Multiple heads increase representational capacity. It is usually too simplistic to assign one fixed human-readable purpose to each head.

5.10 Step 8: Residual Connections

A residual connection adds a sublayer's input to its output:

$$Y = X + F(X),$$

where F may be attention or a feed-forward network. Residual paths help information and gradients flow through deep networks and can make optimisation easier. Modern models differ in the exact arrangement of residual connections and normalisation.

5.11 Step 9: Layer Normalisation

A simplified layer-normalisation equation is

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta,$$

where μ and σ^2 are the mean and variance across the normalised dimensions, γ and β are learned parameters, and ε is a small stabilising constant. Normalisation helps control activation scale and can improve training stability. Exact placement differs across architectures.

5.12 Step 10: Feed-Forward Network

After attention mixes information across token positions, a feed-forward network transforms each token representation:

$$\text{FFN}(x) = W_2\phi(W_1x + b_1) + b_2,$$

where ϕ is a nonlinear activation such as GELU or a gated activation used by some modern models. Attention mainly enables information exchange across positions; the feed-forward sublayer applies learned transformations to the features at each position.

5.13 Step 11: Stacking Blocks

A Transformer stacks multiple blocks. The output of one block becomes the input to the next. A conceptual block contains attention, residual connections, normalisation, a feed-forward network, and another residual connection. This order is illustrative; implementations may use pre-normalisation, gated layers, mixture-of-experts components, or other modifications.

5.14 Step 12: Output Projection and Token Selection

The final hidden state is projected to vocabulary scores called logits:

$$z = hW_{\text{out}} + b_{\text{out}}.$$

For a vocabulary of V tokens, z has V values. Softmax can convert them into probabilities:

$$P(x_{t+1} = i \mid x_{\leq t}) = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}.$$

A decoding method chooses the next token, appends it to the sequence, and repeats until a stopping condition is met. Many implementations use key/value caching to avoid recomputing all earlier attention projections during generation.

5.15 Encoder, Decoder, and Encoder–Decoder

Architecture	Main behaviour	Typical uses
Encoder-only	Builds contextual representations of input; may use bidirectional context	Classification, extraction, representation learning
Decoder-only	Predicts next token using prior context	Text generation, chat, code generation
Encoder–decoder	Encodes input and generates output conditioned on it	Translation, summarisation, text-to-text tasks

These are broad patterns. Architecture alone does not determine every capability.

5.16 Computational Cost

For standard full self-attention over n tokens, the score matrix has n^2 entries. Attention computation and memory therefore grow approximately quadratically with sequence length in a straightforward implementation. Doubling the sequence length can make the attention matrix about four times as large.

Long-context systems use engineering techniques such as efficient attention kernels, sparse attention, grouped-query attention, and key/value caching. Each has trade-offs, and total cost also depends on hidden dimension, feed-forward layers, batch size, hardware, and the number of generated tokens.

6 Pre-training: Learning Broad Patterns

6.1 Definition

Pre-training is a large-scale initial training stage in which a model learns broad patterns from a dataset. For language models, this may include grammar, styles, code patterns, factual associations present in data, and statistical relationships between tokens. The model is not given a hand-written rule for every possible question; its parameters are updated through a learning objective.

6.2 Next-Token Prediction

A common objective for decoder-only language models is next-token prediction. Given "The capital of France is", the model learns probabilities for possible next tokens. "Paris" may receive a high probability if the model learned the relevant pattern.

For a sequence $x_1, \ldots, x_T$:

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1}).$$

A common training loss is negative log-likelihood:

$$\mathcal{L}_{\text{NLL}} = -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}).$$

The model receives a larger penalty when it assigns low probability to the actual next token.

6.3 Other Pre-training Objectives

Not every model uses next-token prediction. Encoder-style models may use masked language modelling, hiding selected tokens and predicting them. Other systems use denoising, contrastive learning, image-text objectives, or combinations. The objective influences what the model learns and how it is used.

6.4 A Simplified Pre-training Pipeline

1. **Data collection:** assemble suitable text, code, or multimodal data under applicable legal and policy constraints.
2. **Preparation:** clean formats, manage duplicates, filter unsuitable content, and address quality and privacy.
3. **Tokenisation:** convert text into token IDs.
4. **Initialisation:** initialise model parameters.
5. **Training:** predict targets, compute loss, backpropagate gradients, and update parameters.
6. **Validation:** monitor held-out performance and training stability.
7. **Evaluation:** assess capabilities, robustness, and failure modes.

Large-scale pre-training can require substantial compute and data engineering. The same ideas can be studied using a small model and dataset.

6.5 What Does the Model Learn?

The model learns parameter values that help with the training objective. Useful behaviour can emerge from data patterns, but the model does not necessarily store facts as a searchable table. Knowledge is represented imperfectly across parameters and activations. A model may reproduce a fact, generalise a pattern, or produce a plausible but incorrect answer. Low training loss does not prove factual reliability.

6.6 Data Quality

More data is not automatically better if it is duplicated, biased, low-quality, unsafe, or unsuitable. Important concerns include provenance, data quality, language coverage, privacy, harmful material, contamination between training and evaluation sets, and legal or licensing constraints.

6.7 Why a Pre-trained Model May Not Be Ready for Users

A base model may be good at continuing text but may not reliably follow instructions. Given "List three advantages of solar energy", it might continue in an unexpected style or format. Instruction tuning and other post-training methods can make it more useful as an assistant. Pre-training builds broad capabilities; it does not automatically create a polished, safe, instruction-following product.

7 Fine-tuning: Adapting a Pre-trained Model

7.1 Definition

Fine-tuning continues training from a pre-trained checkpoint using a more targeted dataset or objective. It can adapt a model to a task, domain, response format, or desired behaviour. It is different from writing a prompt, and it does not guarantee that every new fact will be remembered accurately.

7.2 Supervised Fine-tuning

In supervised fine-tuning (SFT), the dataset contains inputs and desired outputs. The model is trained to produce the target response given the input.

For example:

> **Input:** "Explain the refund policy in two bullet points."
> **Target:** A correct two-bullet summary, based on a curated example.

For an autoregressive model, training often uses next-token cross-entropy on the target response tokens. The examples teach the desired input-output behaviour.

7.3 Preference Optimisation and Alignment

Some post-training methods use preferences or comparisons between candidate responses. Human reviewers or other preference sources may rank responses, and training encourages outputs that satisfy the chosen criteria. Reinforcement Learning from Human Feedback (RLHF) is one family of approaches; other preference-optimisation methods also exist. The exact process differs by model.

These methods can improve instruction-following and preferred response behaviour, but they do not eliminate factual errors or all unsafe outputs.

7.4 Full Fine-tuning

Full fine-tuning updates most or all trainable model parameters. It offers flexibility but may require substantial compute and memory. Poorly designed fine-tuning can overfit or reduce useful capabilities learned during pre-training.

7.5 Parameter-Efficient Fine-tuning and LoRA

Parameter-efficient fine-tuning (PEFT) updates a smaller set of parameters or adds trainable components while keeping most original weights fixed. Low-Rank Adaptation (LoRA) represents a weight update as

$$W' = W + \Delta W, \qquad \Delta W = BA,$$

where W is the original matrix and A, B are smaller learned matrices. A low-rank update can reduce trainable parameter count and storage. Practical LoRA implementations often include a scaling factor. Its effectiveness depends on the model, target layers, dataset, and task.

7.6 Fine-tuning Data Quality

A model learns from the examples provided. Inconsistent labels, incorrect answers, outdated policy examples, or biased decisions can teach undesirable behaviour. A useful dataset should:

- reflect realistic inputs and important edge cases;
- contain accurate, consistent target responses;
- avoid leakage from evaluation data;
- comply with privacy and data-use requirements;
- include separate validation and test procedures.

A smaller curated dataset can be more useful than a larger noisy one.

7.7 Does Fine-tuning Add New Facts?

Fine-tuning can influence a model's responses, terminology, and task behaviour, but it is not always the best way to supply frequently changing facts such as current prices, leave balances, or company policies. For these, retrieving information from a trusted source at inference time may be better. Fine-tuning and RAG can be combined when there is a clear reason.

8 Pre-training vs. Fine-tuning: Direct Comparison

Dimension	**Pre-training**	**Fine-tuning**
Purpose	Learn broad patterns and general capabilities	Adapt a model to a task, domain, or behaviour
Starting point	Usually initialised parameters	A pre-trained checkpoint
Data	Often large and broad datasets	More targeted examples or preference data
Objective	Often next-token prediction or another broad objective	Often supervised task loss or preference-based objective
Compute	Can be extremely expensive at scale	Can be cheaper; full fine-tuning can still be expensive
Outcome	General-purpose base model	Model more adapted to a target use case
Main risks	Data contamination, bias, poor coverage, high cost	Overfitting, inconsistent labels, narrow behaviour, loss of capabilities
Example	Learn general language and code patterns	Learn a consistent support-response format

8.1 A Simple Analogy

Pre-training is like a student studying many subjects and developing a broad foundation. Fine-tuning is like giving that student specialised training for a particular role, such as preparing

tax documents or handling customer-support tickets. The analogy has limits: a model learns through numerical optimisation, not human experience, and its capabilities can be uneven.

8.2 Prompting vs. RAG vs. Fine-tuning

Approach	What changes?	Useful when
Prompting	Instructions and context for a request	Quick adaptation, task framing, and output formatting
RAG	External information retrieved at inference time	Current policies, internal knowledge, catalogues, source-grounded answers
Fine-tuning	Model parameters are further trained	Repeated task patterns, domain behaviour, and consistent formats
Pre-training	Broad initial learning from a large corpus	Building the base capabilities of a model

Start with the simplest approach that meets the requirement. Try prompting and evaluate it. Add RAG when trusted external information is needed. Consider fine-tuning when repeated tests show that model behaviour needs a persistent improvement.

9 Worked Enterprise Example: A Policy Assistant

Suppose a company wants an assistant to answer employee questions about travel reimbursement.

9.1 Possible Architecture

1. The tokenizer converts the question into IDs.
2. The embedding layer maps IDs to vectors.
3. The Transformer processes the contextual representation.
4. A retrieval system searches approved policy documents, if RAG is used.
5. The language model generates an answer conditioned on the question and retrieved passages.
6. Output controls check citations, permissions, formatting, and escalation rules.

9.2 Where Fine-tuning Helps

If the assistant must answer in a fixed structure such as “Eligibility”, “Required documents”, “Approval needed”, and “Source”, supervised fine-tuning may improve format consistency. However, the actual reimbursement limit and current policy version should usually be retrieved from the authoritative policy source rather than relying on the model’s parameters alone.

9.3 Possible Failure Modes

- The tokenizer handles a regional-language query poorly.
- Retrieval finds an outdated policy.

- The model overlooks an exception in a retrieved passage.
- The generated answer invents a reimbursement amount.
- The application returns a document the employee is not authorised to access.

Each failure needs a different response: multilingual evaluation, document versioning, retrieval tests, source constraints, output validation, and backend access control. A model's own assurance that an action is safe is not a replacement for application-level permission checks.

10 Practical Exercises

10.1 Exercise 1: Inspect Tokenisation

Compare token counts for an English sentence, a technical sentence, a Hindi or another Indian-language sentence, a Hinglish sentence, and a Python function. Record the tokenizer used. Do not assume the result generalises to every model.

10.2 Exercise 2: Compare Sentence Embeddings

Try these example queries:

- "How can I claim travel expenses?"
- "What is the reimbursement process?"
- "How do I reset my laptop password?"

Generate embeddings with a suitable sentence-embedding model and compare cosine similarities. Check whether the ranking makes sense. Surprising results are a reason to inspect the model and data, not proof that one similarity measure is always wrong.

10.3 Exercise 3: Draw a Transformer Block

Draw the sequence

$$\text{Input} \to \text{Attention} \to \text{Residual/Norm} \to \text{Feed-forward} \to \text{Residual/Norm} \to \text{Output}.$$

Add query, key, value projections and a causal mask. Explain why the mask is required for next-token training.

10.4 Exercise 4: Choose an Adaptation Method

Imagine a company needs an assistant to answer questions from a policy document that changes monthly. Compare prompting, RAG over the current policy, fine-tuning on example questions, and a combination of RAG and fine-tuning. Discuss freshness, cost, evaluation, and outdated-answer risk.

10.5 Exercise 5: Build an Evaluation Set

Create 20–50 realistic policy questions, including simple cases, ambiguous cases, questions whose answers are missing, conflicting document versions, and restricted material. Record the expected answer or behaviour, source, and scoring criteria. Keep this set separate from training examples.

11 Common Misunderstandings

1. **One token equals one word.** Not necessarily; tokens can be subwords or punctuation.
2. **An embedding is a human-readable list of meanings.** Usually not; features are learned and distributed across dimensions.
3. **Attention weights explain all model reasoning.** No; they are one component of a multi-layer computation.
4. **Pre-training guarantees factual answers.** No; it learns statistical patterns and can still produce unsupported claims.
5. **Fine-tuning is always better than prompting.** No; it adds data, compute, and maintenance requirements.
6. **Fine-tuning is always the best way to update facts.** Frequently changing facts are often better retrieved from a maintained source.
7. **A large context window solves every long-document problem.** Retrieval quality, cost, computation, and effective use of context still matter.
8. **English performance guarantees equal performance in Indian languages.** This must be measured, not assumed.

12 Glossary

Activation
Intermediate value produced by a neural-network layer.

Attention
Mechanism for combining information from token positions using learned weights.

Context window
Amount of tokenised context a model can process in one interaction.

Embedding
Learned vector representation of a token, sentence, document, image, or other item.

Fine-tuning
Further training of a pre-trained model on a targeted dataset or objective.

Gradient
Derivatives indicating how a function changes with its parameters.

Hidden state
Intermediate representation inside a neural network.

Hyperparameter
A training or architecture setting selected outside parameter updates.

Key
Representation used to calculate how relevant a token is to a query.

Logit
An unnormalised score that can be converted to probability by softmax.

LoRA
Parameter-efficient method that learns low-rank weight updates.

Parameter
Learned numerical value in a model.

Pre-training
Initial broad training to learn patterns and capabilities.

Query
Representation used to compare a token with keys from other tokens.

RAG
Retrieval-Augmented Generation: retrieve external information and supply it to a model at inference time.

Residual connection
Adds a sublayer's input to its output.

Softmax
Converts scores into non-negative values that sum to one.

Token
Discrete unit of text or another modality processed by a model.

Tokenizer
Converts text into tokens and IDs, and can decode IDs into text.

Value
Representation combined by attention using its weights.

13 Self-Assessment Questions

1. Why must text be tokenised before a standard language model can process it?
2. What is the difference between a token and a token ID?
3. Why are subword tokenizers common?
4. How can tokenisation affect cost and context length?
5. How does a contextual representation differ from a token embedding?
6. What does cosine similarity measure, and what does it not guarantee?
7. What are the roles of queries, keys, and values?
8. Why is the attention score divided by $\sqrt{d_k}$?
9. What does softmax do?
10. Why does a GPT-style model use a causal mask?
11. What do residual connections and layer normalisation contribute?
12. What is the role of the feed-forward network?
13. Compare encoder-only, decoder-only, and encoder–decoder architectures.
14. Explain next-token prediction and the pre-training loss.

15. How does supervised fine-tuning differ from pre-training?
16. What is LoRA, and why can it reduce fine-tuning cost?
17. When is RAG a better option than fine-tuning?
18. Why should test examples be separate from training examples?
19. Name three failure modes in an enterprise Transformer application.
20. What should be measured before deploying an assistant to employees?

14 Conclusion

Tokenisation, embeddings, Transformers, pre-training, and fine-tuning are connected ideas behind many modern language-model systems. Tokenisation converts text into discrete IDs. Embedding layers turn those IDs into vectors. Transformer blocks use attention and feed-forward transformations to build contextual representations. Pre-training learns broad patterns, while fine-tuning adapts a model to specific tasks or desired behaviours.

The practical lesson is to choose components according to the problem. A model architecture does not guarantee correctness; an embedding does not guarantee relevant retrieval; and fine-tuning does not guarantee fresh knowledge. Dependable applications combine suitable models with trustworthy data, evaluation, access controls, and human oversight.

References

[1] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017. https://arxiv.org/abs/1706.03762

[2] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," 2018. https://arxiv.org/abs/1810.04805

[3] A. Radford et al., "Improving Language Understanding by Generative Pre-Training," OpenAI, 2018. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

[4] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," 2020. https://arxiv.org/abs/1910.10683

[5] E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," 2021. https://arxiv.org/abs/2106.09685

[6] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing," 2018. https://arxiv.org/abs/1808.06226

[7] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020. https://arxiv.org/abs/2005.11401

[8] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," 2022. https://arxiv.org/abs/2203.02155